COMP 264 FINAL PROJECT REPORT

sara hamoleila- Teawo Kim-Syed Hussain

COMP264 Group 4

Table of Contents

1. Rational and Scope 4
1.1 Problem Definition 5
1.2 Proposed Solution 5
1.3 Benefits 5
1.4 Scope 5
2. Research 5
2.1 Market Analysis 6
2.2 Literature Review 6
3. System Design 6
3.1 Architecture Overview 6
3.2 Use Case Diagram 7
3.3 Component Design 7
3.4 Interaction Flow 8
3.5 User Interface Design 9
4. Implementation 10
4.1 Backend 10
4.2 API Endpoints 11
4.3 Data Flow 12
5. Testing 12
5.1 Test Images Used 12
5.2 Results 13
5.3 AWS Verification 14
6. Future Work 15
7. Conclusion 16
References 16

Universal AI-Powered Product Identification and Multilingual Catalog Generator

1. Rational and Scope

1.1 Problem Definition

Small and medium businesses face challenges when listing products online, including:

- Manual product description generation
- Language barriers for global markets
- Time-consuming catalog creation

1.2 Proposed Solution

This project implements a serverless AI-powered web application that:

- Accepts product images
- Automatically identifies objects

- Generates multilingual labels

1.3 Benefits

- Automation of product listing
- Reduced manual effort
- Multilingual support (Korean, Spanish, French)
- Scalable cloud-based architecture

1.4 Scope

The system includes:

- Image upload (base64 via frontend)
- Object detection using AWS Rekognition
- Translation using AWS Translate
- Storage using DynamoDB
- Image hosting using S3
- REST API using AWS Chalice

2. Research

2.1 Market Analysis

Tool	Functionality	Limitation
Google Lens	Image recognition + translation	Not customizable
Shopify Magic	Product descriptions	Limited to Shopify
Pinterest	Visual search	No multilingual focus

2.2 Literature Review

Recent advancements in cloud-based artificial intelligence services have enabled scalable and efficient image recognition systems. According to Deng et al. (2009), large-scale image datasets such as ImageNet have significantly improved object detection accuracy using deep learning models. These models form the foundation of modern computer vision systems, including services like AWS Rekognition.

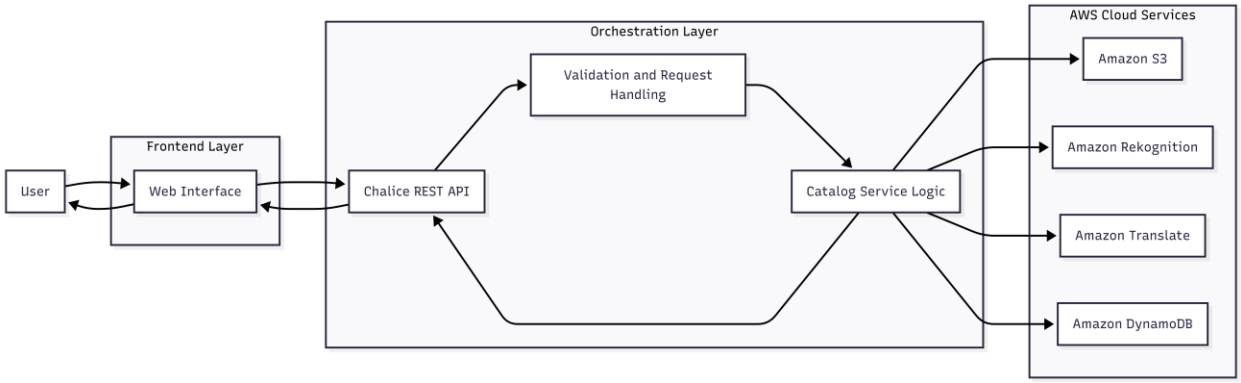
In addition, Hossain et al. (2019) highlight that integrating AI services with cloud computing allows developers to build intelligent applications without the need for complex model training. This aligns with the approach used in this project, where AWS Rekognition and AWS Translate are utilized to provide automated product identification and multilingual support.

These studies support the effectiveness of using pre-trained AI models and cloud infrastructure to build scalable, real-world applications.

3. System Design

3.1 Architecture Overview

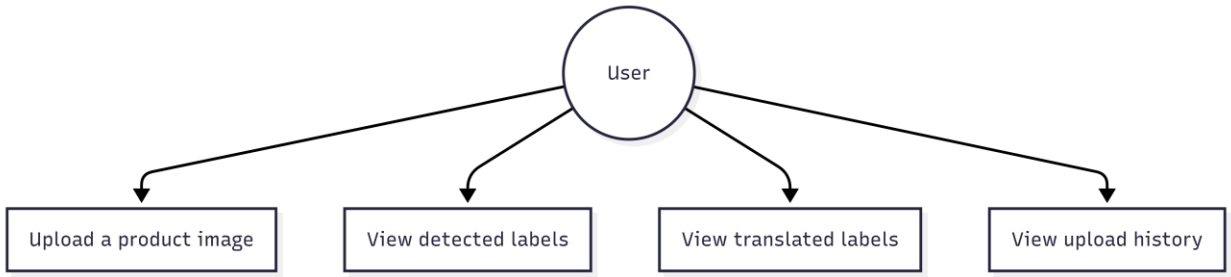
The proposed system follows a serverless architecture that integrates a web-based frontend with multiple AWS cloud services. The system is designed using a modular structure where each component performs a specific function, ensuring scalability, flexibility, and maintainability.



As illustrated in Figure 1, the user interacts with the system through the frontend interface. The request is sent to the Chalice REST API, which processes the request through the catalog service layer. The image is stored in Amazon S3, analyzed using Amazon Rekognition, translated using Amazon Translate, and the results are stored in DynamoDB before being returned to the user.

3.2 Use Case Diagram

The use case diagram illustrates the interactions between the user and the system. It highlights the main functionalities provided by the application and shows how the user engages with different features of the system.



As shown in Figure 2, the user can upload a product image, view detected labels, view translated labels in multiple languages, and access the upload history. These use cases represent the core functionalities of the system and define the primary interactions between the user and the application

3.3 Component Design

Backend Entry (Chalice App)

- Routes defined in app.py
- Handles:
    - /upload
    - /analyze
    - /results

Configuration Layer

- Loads environment variables
- Controls:
    - AWS region
    - S3 bucket
    - DynamoDB table
    - target languages

Service Layer (Core Logic)

Catalog Service (Main Orchestrator)

- Coordinates all services
- Steps:
    - Decode image
    - Upload to S3
    - Detect labels
    - Translate labels
    - Store record

S3 Service

- Uploads images
- Generates image URL

Rekognition Service

- Detects labels
- Includes fallback logic

Translate Service

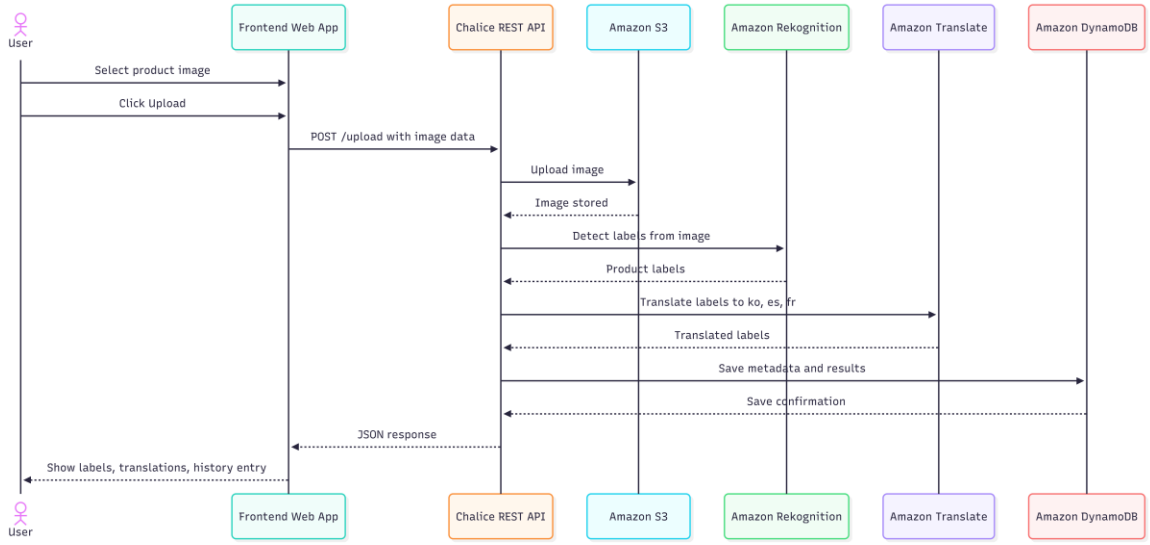
- Translates into:
  - Korean
  - Spanish
  - French

DynamoDB Service

- Stores metadata
- Supports:
  - save
  - retrieve
  - list

3.4 Interaction Flow

Several design decisions were made to ensure system efficiency and scalability. The use of a serverless architecture reduces infrastructure management overhead. AWS Rekognition was selected to avoid the need for custom model training, while DynamoDB provides fast and scalable NoSQL storage.

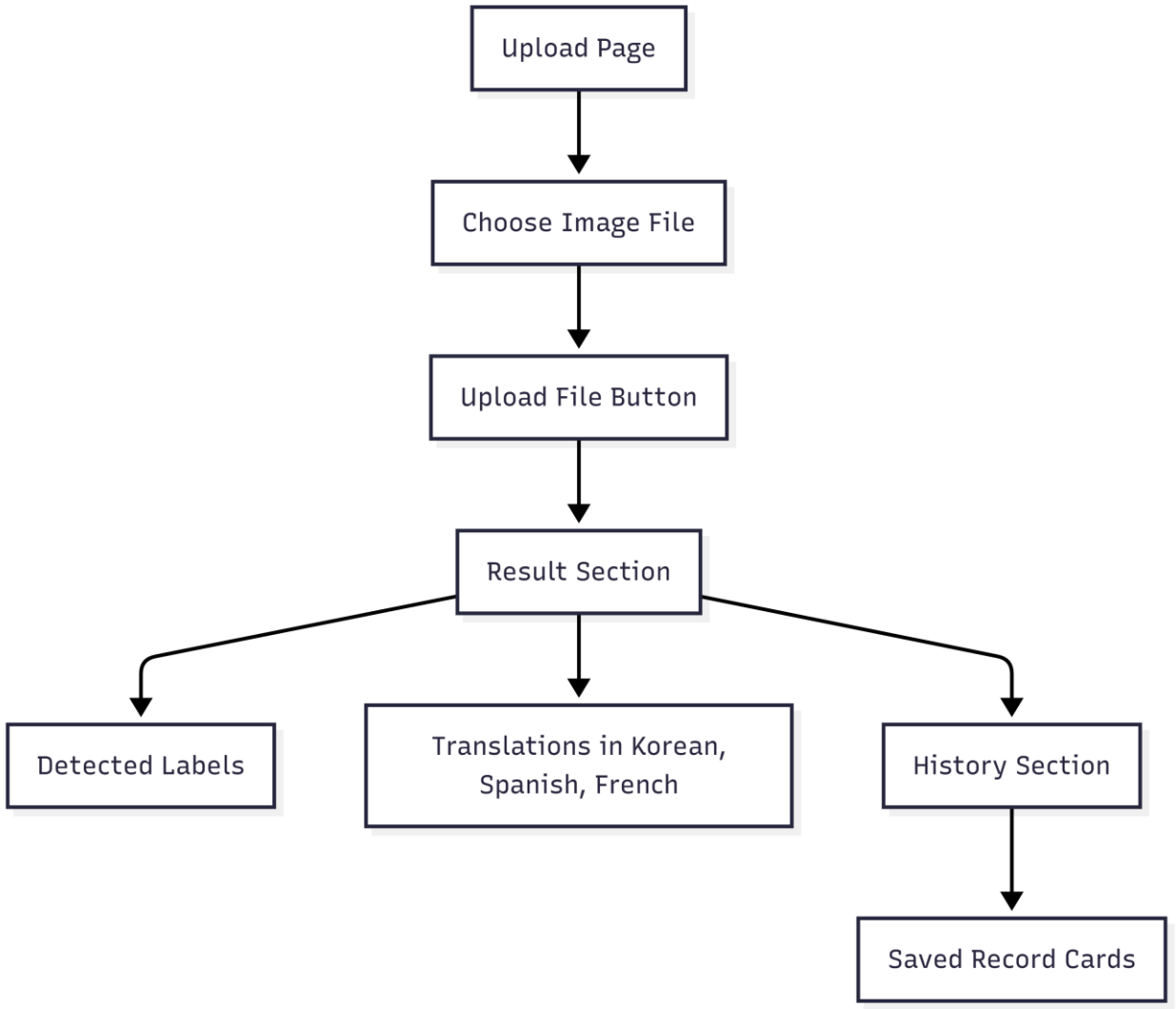


As shown in Figure 2, the user initiates the process by selecting and uploading an image through the frontend application. The frontend sends a POST request to the Chalice REST API, which processes the request by uploading the image to Amazon S3. The

system then calls Amazon Rekognition to detect product labels, followed by Amazon Translate to generate multilingual labels. The results are stored in DynamoDB, and a JSON response is returned to the frontend, where the user can view the detected labels, translations, and history.

3.5 User Interface Design

The user interface was designed to provide a simple and intuitive workflow for uploading images and viewing results. The interface guides the user through a clear sequence of steps, ensuring ease of use and efficient interaction with the system.

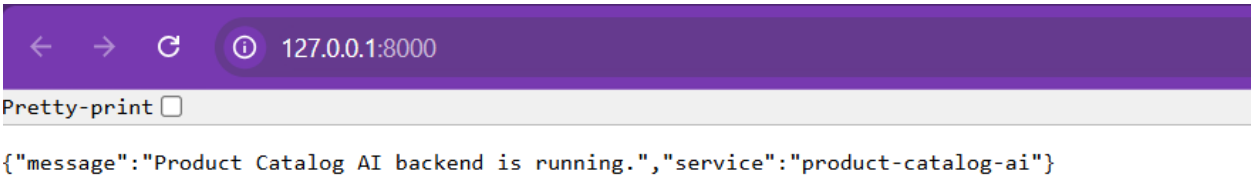


As shown in Figure 3, the user begins by accessing the upload page and selecting an image file. After clicking the upload button, the system processes the image and displays the results, including detected labels and translated outputs. Additionally, a history section allows users to view previously processed items, improving usability and data accessibility.

4. Implementation

4.1 Backend

- Framework: AWS Chalice
- Language: Python
- Structure:
    - Modular services
    - Separation of concerns



Backend Service Verification

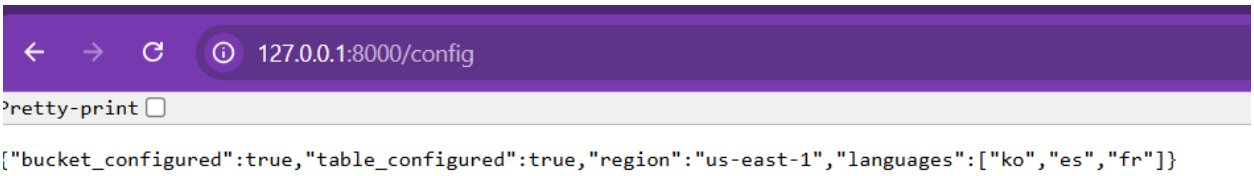
The backend service was successfully implemented using AWS Chalice.

The application was tested locally using the Chalice development server.

Accessing the root endpoint (http://127.0.0.1:8000) confirms that the backend API is running and ready to process requests.

4.2 API Endpoints

Endpoint	**Function**
/health	Check system
/config	Show configuration
/upload	Process image
/results	Fetch history



Configuration Verification

The configuration endpoint was used to verify that all AWS resources were correctly connected. The system confirms that the S3 bucket and DynamoDB table are properly configured, and the application is running in the correct AWS region. This ensures that the backend can successfully interact with AWS services during execution.

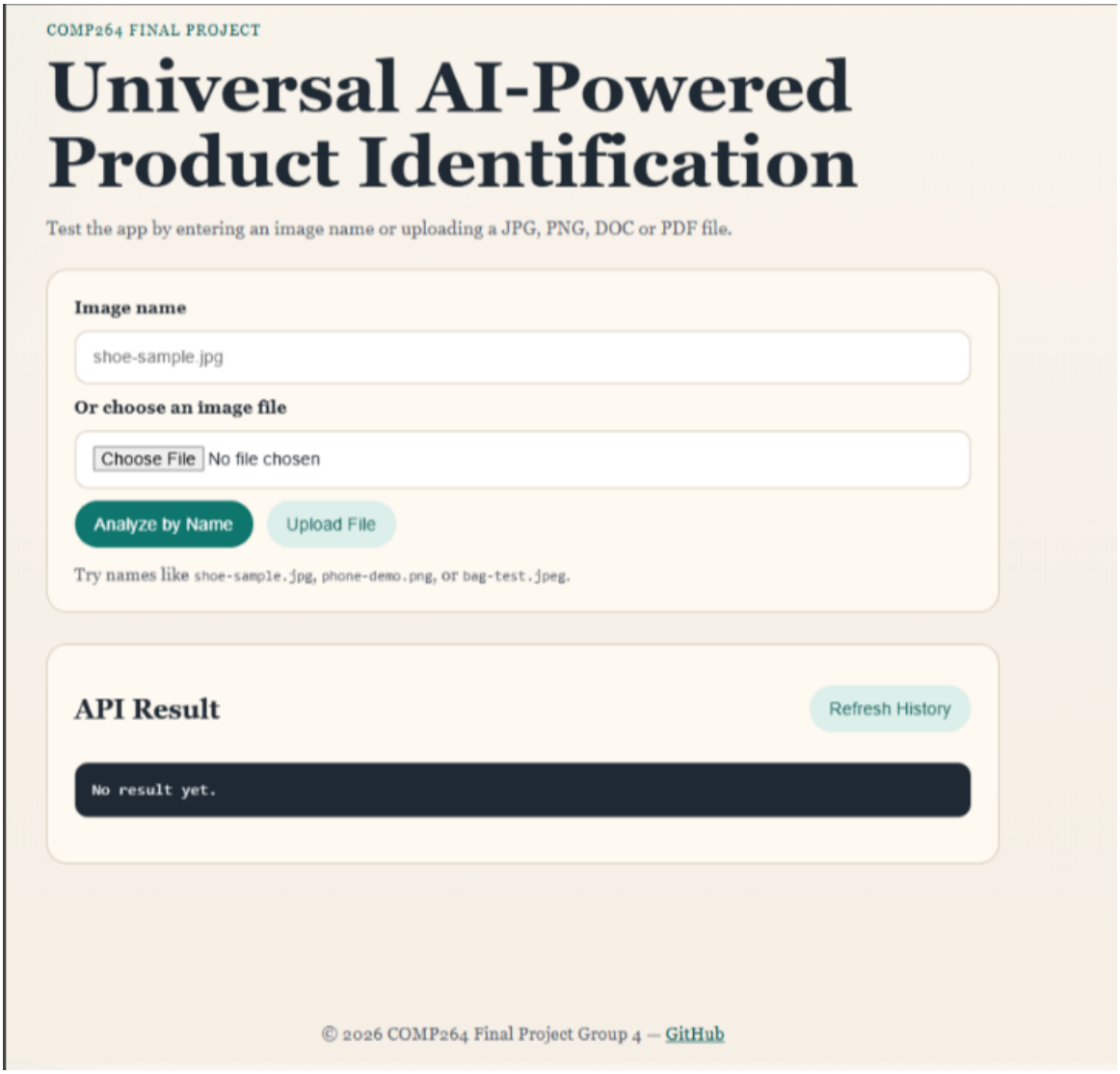
4.3 Data Flow

1. User uploads image
2. Image converted to base64
3. Backend decodes image
4. Image uploaded to S3
5. Rekognition detects labels
6. Translate converts labels
7. Data stored in DynamoDB
8. Response returned

4.4 Supported File Formats

The system is designed to process image files such as JPEG and PNG. Additionally, the application can be extended to support document formats such as PDF and DOCX by extracting embedded images or text for processing using AWS services.

This enhances the system's flexibility and allows it to handle real-world product catalogs stored in document formats.



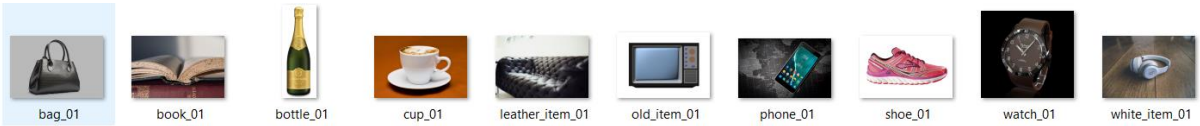
5. Testing

5.1 Test Images Used

tested with:

- Bag
- Book
- Bottle
- Coffee cup

- Sofa
- Phone
- Shoe
- Watch
- Headphones



5.2 Results

The system was tested using multiple product images, including bags, bottles, and electronic items. The application successfully processed all inputs and produced accurate outputs. As shown in Figure X, the system correctly detected product labels using AWS Rekognition and generated multilingual translations using AWS Translate. The processed image was uploaded to Amazon S3, and the results were stored in DynamoDB.

The output confirms that the system is fully functional, operating in AWS mode rather than fallback mode. This demonstrates successful integration of all cloud services, including S3, Rekognition, Translate, and DynamoDB.

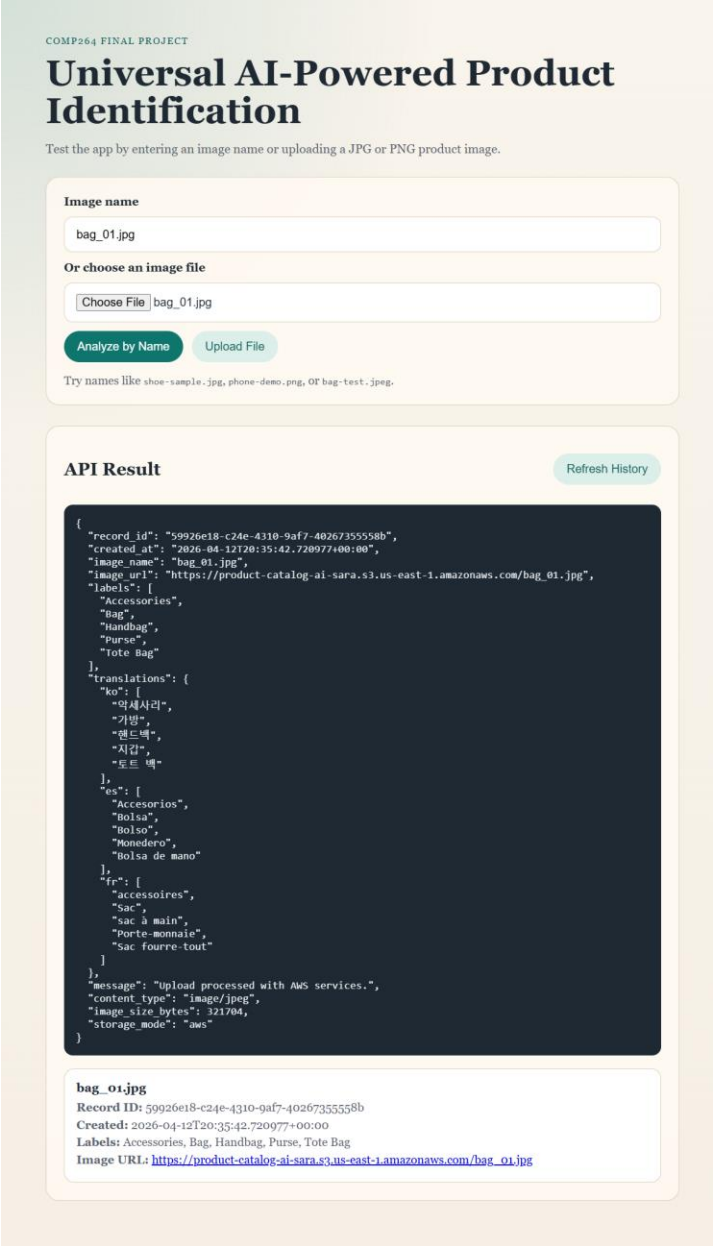


Figure X: Successful product identification and multilingual output using AWS services

5.3 AWS Verification

The system was verified using AWS Console to ensure proper integration with cloud services.

As shown in Figure X, product images were successfully uploaded and stored in Amazon S3. The presence of uploaded files confirms that the system correctly handles image storage as part of the processing pipeline.
Additionally, the system stores processed metadata in DynamoDB, ensuring persistent storage and retrieval of results.

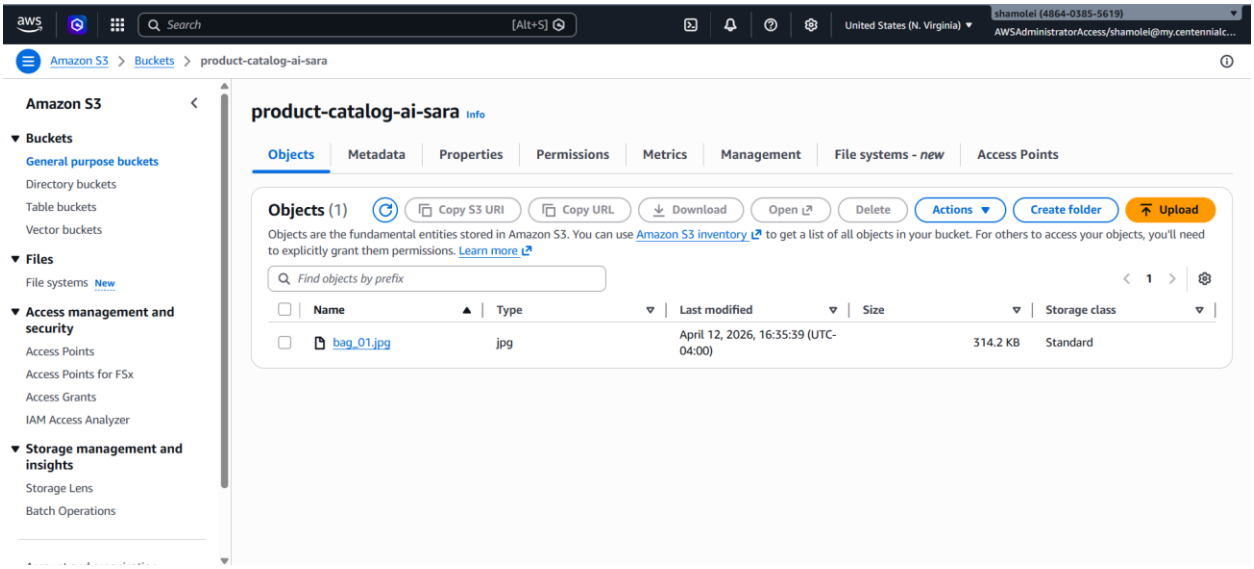


Figure X: Images successfully stored in Amazon S3 bucket

Additionally, DynamoDB was used to store metadata for each processed image. As shown in Figure X, the database contains records including detected labels, translated outputs, and image URLs. This confirms that the system successfully stores and retrieves structured data using a NoSQL database.

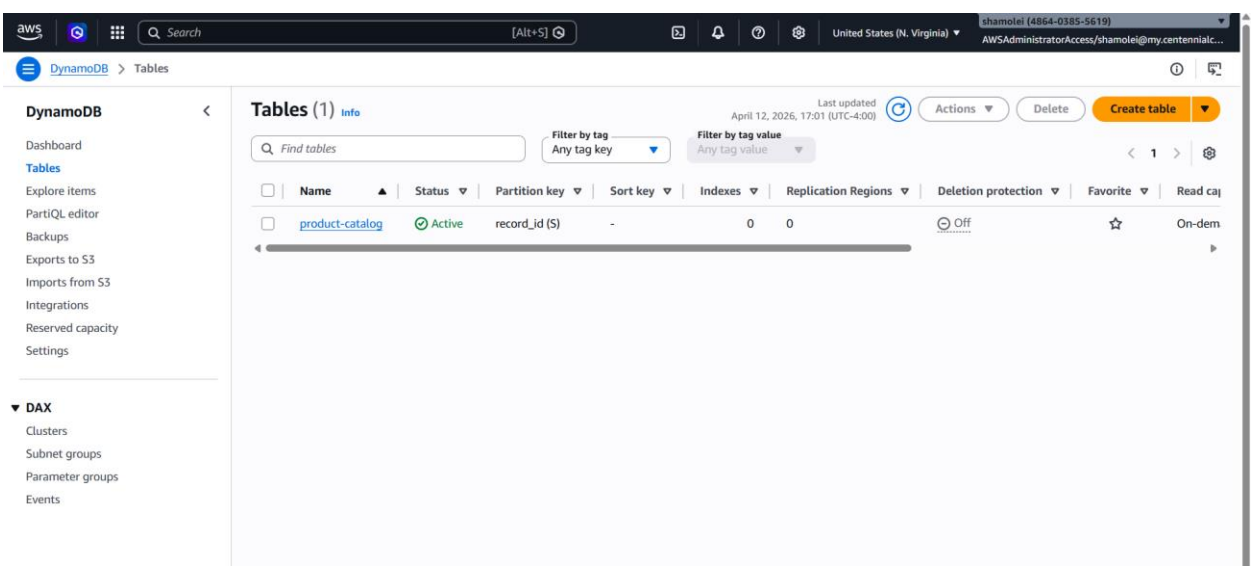


Figure X: Stored product metadata in DynamoDB table

6. Future Work

- Add product recommendation system

- Improve detection accuracy
- Add pricing estimation
- Deploy publicly

7. Conclusion

The system demonstrates a complete **serverless AI application** that:

- integrates multiple AWS AI services
- processes real-world data
- provides scalable and efficient solution

References

Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009).

ImageNet: A large-scale hierarchical image database.

https://doi.org/10.1109/CVPR.2009.5206848

Hossain, M. S., Muhammad, G., & Alamri, A. (2019).

Smart healthcare monitoring: A voice pathology detection paradigm for smart cities.

Future Generation Computer Systems, 101, 30–38.

https://doi.org/10.1016/j.future.2019.06.022